



Starter Cashback Engine PRD

MCOM Platform — Internal Cashback & Booking Incentive System

1. Document Overview

Purpose

This document defines the product requirements for the **Starter Cashback Engine**, an internal reward system designed to:

- Incentivize customer behavior
- Drive bookings and purchases
- Increase repeat engagement
- Protect business revenue
- Keep reward value circulating inside the platform

This cashback engine is **not traditional bank cashback**. It is a controlled, internal credit system used to reduce prices, unlock offers, and influence user behavior.

2. Core Concept

The Starter Cashback Engine is a **platform-controlled credit system** that rewards users with internal value when they perform specific actions.

Key characteristics:

- Cashback exists only inside the platform
- Cannot be withdrawn to a bank account
- Can only be used for purchases or bookings
- Fully rule-controlled by the platform

Think of it as:

“Smart discount credit that keeps customers active inside the ecosystem.”

3. Cashback Types

3.1 Internal Cashback (Primary System)

This is the starter engine covered in this PRD.

Behavior:

- User earns cashback after eligible actions
- Cashback appears in dashboard wallet
- Cashback reduces future payments
- Cashback never leaves platform

Allowed uses:

- Service bookings
- Product purchases
- Campaign promotions

Restrictions:

- Cannot withdraw
 - Cannot transfer
 - Cannot convert to cash
-

3.2 Traditional Cashback (Out of Scope)

Real-money cashback exists in other platform modules and is **not part of this starter engine**.

4. System Goals

The engine must:

- Reward engagement
 - Encourage repeat spending
 - Reduce booking friction
 - Fill idle time slots
 - Create loyalty loops
 - Protect business margins
-

5. User Journey — Step-by-Step Cashback Flow

Step 1 — Trigger Action

User performs an eligible action:

Examples:

- Books a service

- Purchases a product
- Wallet top-up promotion
- Campaign participation

System checks eligibility rules.

Step 2 — Cashback Calculation

Engine calculates reward based on rules:

Possible rule types:

- Percentage cashback
- Fixed cashback value
- Tier-based reward
- Campaign override

Example:

Purchase = £100 Cashback rule = 10% Reward = £10 internal credit

Step 3 — Credit Issuance

System credits cashback to user wallet:

Wallet shows:

- Cashback balance
- Earn history
- Expiration status

User receives notification.

Step 4 — Cashback Storage Rules

Cashback is stored with metadata:

- Source action
 - Issued timestamp
 - Expiry date
 - Usage restrictions
 - Campaign tags
-

Step 5 — Cashback Redemption

During checkout or booking:

User can:

- Apply cashback fully
- Apply partially
- Save for later

System enforces limits:

- Max redemption percentage
 - Business restrictions
 - Campaign scope
-

Step 6 — Payment Resolution

Final price calculation:

Original price – Cashback applied = Amount payable

System records transaction breakdown.

Step 7 — Lifecycle Completion

Cashback transitions:

Issued → Available → Redeemed → Expired

Audit logs maintained.

6. Booking Integration Flow

The cashback engine integrates directly with service booking.

Booking sequence:

1. User selects service slot
2. System shows price
3. Cashback eligibility displayed
4. User applies cashback
5. Booking confirmed

6. Reward loop continues

Optional rules:

- Cashback-only booking promotions
 - Off-peak incentives
 - Loyalty booking credits
-

7. Merchant Controls

Businesses can configure:

- Cashback percentage
- Campaign eligibility
- Redemption limits
- Service inclusion/exclusion

Dashboard must show:

- Cashback impact analytics
 - Redemption trends
 - Booking influence metrics
-

8. Platform Controls

Admin system controls:

- Global cashback rules
- Campaign overrides
- Fraud prevention thresholds
- Expiry policies
- Reward caps

Admin visibility:

- Cashback issuance logs
 - Redemption logs
 - Abuse monitoring
-

9. Wallet Design Requirements

Wallet interface must display:

- Total cashback balance
- Pending rewards
- Expiring credits
- Usage history

Clear messaging:

"Internal cashback — usable only inside platform."

10. Rule Engine Architecture

Cashback engine must support rule layering:

Priority order:

1. Platform campaign rule
2. Merchant override
3. Default cashback rule

Conflict resolution required.

11. Expiry Logic

Cashback may expire based on rules:

Examples:

- 30-day promotional credit
- Seasonal reward expiry

System behavior:

- Notify user before expiry
 - Remove expired credit automatically
-

12. Security & Abuse Protection

Required safeguards:

- Duplicate reward prevention
 - Suspicious activity detection
 - Redemption throttling
 - Audit trails
-

13. Analytics & Reporting

Metrics tracked:

- Cashback issued volume
- Redemption rate
- Booking conversion uplift
- Repeat customer rate

Merchant reports:

- Reward ROI
 - Booking influence
-

14. Edge Cases

System must handle:

- Partial refunds
- Booking cancellation
- Cashback reversal
- Expired credits during checkout

Clear rollback logic required.

15. Notifications

User notifications include:

- Cashback earned
- Cashback redeemed
- Expiry reminder

Channels:

- In-app
 - Email
 - Push alerts
-

16. MVP Scope

Included:

- Internal cashback wallet
- Booking redemption
- Basic rule engine
- Merchant dashboard controls
- Admin monitoring

Excluded:

- Real-money cashback
 - Cross-platform transfers
 - AI reward prediction
-

17. Future Enhancements

Planned expansions:

- Dynamic cashback targeting
 - AI reward optimization
 - Gamified loyalty tiers
 - Partner reward pools
-

18. Success Criteria

- Increased booking conversion rate
 - Higher repeat engagement
 - Reduced idle booking slots
 - Merchant adoption
-

19. Risks & Mitigation

Risk	Impact	Mitigation
Reward abuse	Revenue loss	Rule throttling
User confusion	Support burden	Clear wallet messaging
Merchant misuse	Margin erosion	Admin oversight

20. Open Questions

- Default cashback percentages?
 - Expiry duration standards?
 - Merchant override limits?
 - Promotion stacking rules?
-

End of Document